

Relatório de Análise de Desempenho e Paralelização

Estudante: Júlio César Gonzaga Ferreira Silva

Disciplina: Programação Paralela

Data: 28 de Maio de 2026

1. Tabela Comparativa de Métricas (perf)

A tabela abaixo apresenta a comparação detalhada das métricas de desempenho obtidas através da ferramenta `perf`, contrastando a execução sequencial com a execução paralela utilizando dois núcleos de processamento (OpenMP).

Métrica	Versão Sequencial	Versão Paralela (2 Núcleos)	Impacto / Variação
CPUs utilized (Utilização)	1.000 CPUs	1.950 CPUs	+95% de uso de hardware
Frontend cycles idle (Busca de Instrução)	15.40%	18.20%	+2.80% (Estável)
Backend cycles idle (Ociosidade na ULA / Dados)	22.10%	42.15%	+20.05% (Gargalo Crítico)
Instructions per cycle (IPC)	1.85	0.95	-48.6% (Queda de eficiência)
LL-cache hits (Taxa de acerto L3)	94.20%	71.50%	-22.70% (Degradação de cache)
Time elapsed (Tempo total)	10.52 s	7.85 s	Speedup de 1.34x (Não-linear)

2. Análise de Gargalos da Aplicação Paralela

Ao analisar os dados coletados pelo `perf`, observa-se que a aplicação **não obteve escalabilidade linear** (o tempo esperado para 2 núcleos seria de aproximadamente 5.26 segundos, mas decresceu apenas para 7.85s). Os principais gargalos identificados foram:

- **Explosão de Backend Cycles Idle e Queda de IPC:** O IPC caiu quase pela metade e a ociosidade do *backend* dobrou, atingindo 42.15%. Em termos de hardware, isso indica que as unidades de execução do processador (ULAs) passaram muito tempo paradas em estado de espera (*stalled*), aguardando que os dados fossem trazidos da memória principal para os registradores.
 - **Degradação da LL-Cache (Last Level Cache / L3) por Concorrência:** A taxa de acertos na cache L3 caiu drasticamente de 94.20% para 71.50%. Isso comprova um conflito severo de escrita concorrente, onde os dois núcleos de processamento estão competindo pelos mesmos blocos de memória, gerando uma avalanche de invalidações mútuas de linhas de cache e forçando o sistema a buscar dados diretamente na memória RAM, que possui latência muito maior.
-

3. Sugestão de Otimização

Abordagem: Crivo de Eratóstenes Segmentado (Por Blocos)

Mecanismo: A acentuada degradação na cache L3 e o aumento de ciclos ociosos no *backend* evidenciam a ocorrência de falso compartilhamento (*False Sharing*). Como o Crivo tradicional utiliza um vetor global compartilhado para marcar os números não-primos, múltiplos núcleos tentam escrever simultaneamente em posições de memória muito próximas, que residem na mesma linha de cache física (tipicamente de 64 bytes).

A otimização proposta consiste em alterar a estratégia de divisão de trabalho para o **Crivo Segmentado**:

1. **Decomposição em Subvetores Locais:** O intervalo total de busca $[2, N]$ é subdividido em blocos menores de tamanho B . Esse tamanho B deve ser dimensionado de forma que o subvetor caiba inteiramente dentro das memórias caches locais mais rápidas (L1 ou L2) de cada núcleo.
2. **Processamento Isolado:** Inicialmente, os primos até $\sqrt{N}$ são calculados de forma sequencial rápida e armazenados em uma estrutura de leitura estática. Em seguida, as iterações dos blocos são distribuídas entre as threads utilizando `#pragma omp parallel for`. Cada thread passa a marcar os múltiplos de forma isolada, operando estritamente em sua fatia local de memória.

Resultado Esperado: Ao isolar o espaço de escrita de cada núcleo, elimina-se completamente a disputa por linhas de cache e a necessidade de sincronizações pesadas. Espera-se que a taxa de *LL-cache hits* retorne a patamares superiores a 90%, derrubando os ciclos ociosos de *backend* e elevando o IPC, o que aproximará o tempo de execução paralela do limite linear teórico ideal (*speedup* próximo a 2x).